

Laboratorio di programmazione per sistemi ciberfisici

17. Alberi

Enrico Martini

May 4, 2026

Definizione

Un albero è una struttura dati non lineare composta da nodi collegati tra loro in modo gerarchico. È un caso particolare di **grafo aciclico connesso**.

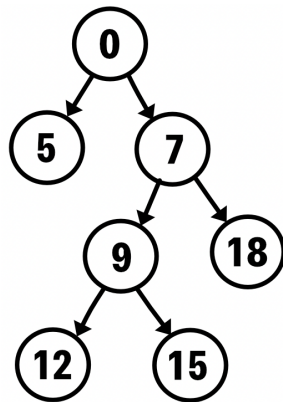
Terminologia:

- ▶ **Radice (Root)**: Il nodo principale da cui "parte" tutto l'albero.
- ▶ **Figli (Children)**: Nodi direttamente collegati e discendenti da un nodo.
- ▶ **Padre (Parent)**: Il nodo superiore diretto di un dato nodo.
- ▶ **Foglia (Leaf)**: Nodo senza figli.
- ▶ **Altezza**: Distanza massima in termini di numero di archi tra la radice e una foglia.
- ▶ **Profondità**: La distanza tra la radice e un nodo specifico.
- ▶ **Grado**: Numero di figli di un nodo.

Alberi Binari

A differenza di alberi generici, in un albero binario ogni nodo ha al massimo due figli, comunemente chiamati **left child** (figlio sinistro) e **right child** (figlio destro).

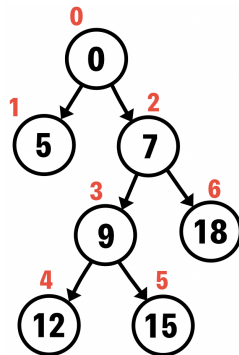
```
1  struct Node {  
2      int data;  
3      struct Node* left;  
4      struct Node* right;  
5  };
```



Pre-Order Traversal (Visita Anticipata)

La visita anticipata è utile per **serializzare l'albero**. Segue l'idea *"radice prima, poi figli"*.

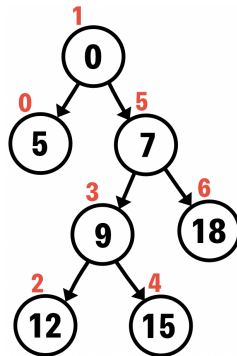
```
1 void preOrder(Node* root) {  
2     if (root != NULL) {  
3         printf("%d ", root->data);  
4         preOrder(root->left);  
5         preOrder(root->right);  
6     }  
7 }
```



In-Order Traversal (Visita Simmetrica)

La visita simmetrica restituisce i nodi dalla foglia sx alla foglia dx.

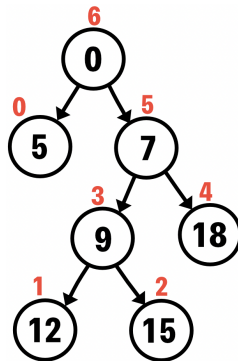
```
1 void inOrder(Node* root) {  
2     if (root != NULL) {  
3         inOrder(root->left);  
4         printf("%d ", root->data);  
5         inOrder(root->right);  
6     }  
7 }
```



Post-Order Traversal (Visita Posticipata)

Molto utile per eliminare alberi, liberando la memoria in modo sicuro.

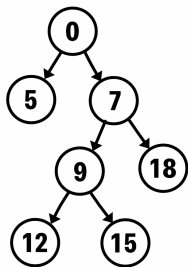
```
1 void postOrder(Node* root) {  
2     if (root != NULL) {  
3         postOrder(root->left);  
4         postOrder(root->right);  
5         printf("%d ", root->data);  
6     }  
7 }
```



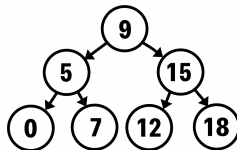
Alberi Binari di Ricerca (BST)

Sono alberi binari che seguono la proprietà di ordinamento: *Dato un nodo dal valore N , tutti i nodo del sottoalbero sinistro contengono valori minori di N , mentre tutti i nodi del sottoalbero destro contengono valori maggiori di N .*

Albero binario:



Albero binario di ricerca:



Esercizio 1

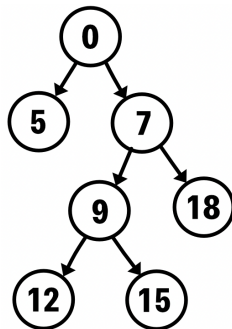
Scrivere un programma che implementi un albero binario e una funzione per stamparlo a video.

Esercizio 2

Scrivere un programma che implementi un albero binario e una funzione ricorsiva che conti il numero totale di nodi.

Esempio:

```
1 nodi: 7
```

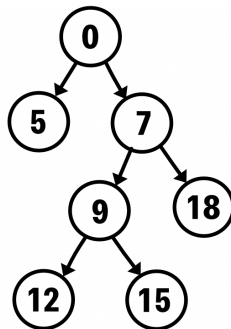


Esercizio 3

Scrivere un programma che implementi un albero binario e una funzione ricorsiva che conti il numero totale di foglie.

Esempio:

```
1 foglie: 4
```

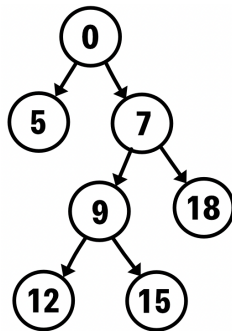


Esercizio 4

Scrivere un programma che implementi un albero binario e una funzione ricorsiva che ne calcoli l'altezza.

Esempio:

```
1 altezza: 3
```

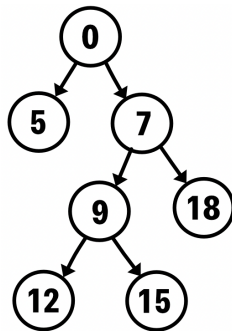


Esercizio 5

Scrivere un programma che implementi un albero binario e una funzione ricorsiva che calcoli la somma di tutti i valori dei nodi.

Esempio:

```
1 somma: 66
```

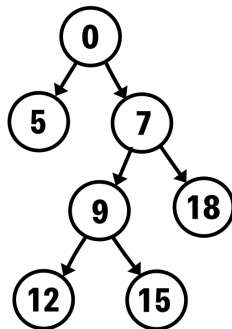


Esercizio 6

Scrivere un programma che implementi un albero binario e una funzione ricorsiva che ritorni il valore minimo.

Esempio:

```
1 minimo: 0
```

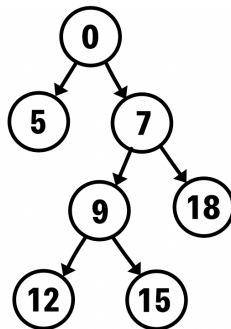


Esercizio 7

Scrivere un programma che implementi un albero binario e una funzione ricorsiva che ritorni la foglia dal valore massimo.

Esempio:

```
1 massima foglia: 18
```

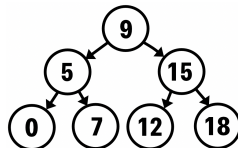


Esercizio 8

Scrivere un programma che implementi un BST.

Input:

```
1 9 5 0 7 15 18 12
```



Esercizio 9

Scrivere un programma che implementi un BST e una funzione ricorsiva che cerchi se un valore esiste nel BST.

Esercizio 10

Scrivere un programma che implementi un BST e una funzione ricorsiva che trovi il valore massimo del BST.

Esercizio 11

Scrivere un programma che implementi un BST e una funzione ricorsiva che trovi il valore minimo del BST.

Esercizio 12

Scrivere un programma che implementi un BST e una funzione ricorsiva stampi i numeri contenuti in un BST in ordine crescente.

Esercizio 13

Scrivere un programma che implementi un BST e una funzione ricorsiva stampi i numeri contenuti in un BST in ordine decrescente.

Esercizio 14

Scrivere un programma che implementi un BST e una funzione che ritorni 1 se l'albero è bilanciato.

NB: Un albero binario è bilanciato se, per ogni nodo, la differenza di altezza tra i due sottoalberi è al massimo 1.